

I. CONCLUSION

A. Project Results

This project successfully demonstrated the design and implementation of a low-cost, open-source SCADA and DCS system using a Raspberry Pi and an Arduino. By integrating these consumer-grade components, the system effectively recreated the core functions of commercial industrial automation platforms. The final architecture proved to be a highly accessible alternative to traditional PLC-based setups. Successful management of hardware and software constraints led to fast development and integration. The Raspberry Pi SCADA supervisor provided user friendly CLI and GUI interfaces over local Wi-Fi, along with dynamic code implementation capabilities. Operation was reliable and the Arduino based DCS successfully maintained a user-specified water level within a tank system by interpreting analog water level sensor data and actuating 12V DC pumps and solenoid valves.

Throughout development, several critical engineering challenges were navigated and resolved. Initial power distribution and gate drive issues that hindered reliable relay switching were solved by replacing discrete MOSFET circuits with a commercial relay module featuring optocoupler isolation. Furthermore, control code instability caused by water ripple was effectively controlled by implementing a moving average function and an incrementing counter to stabilize sensor feedback. By incorporating safety redundancies like manual overrides and emergency dump valves, the design adhered to ISA-112 and IEEE recommended practices for system reliability. Ultimately, this project proves that while consumer-grade hardware falls short of the complexity needed in industrial use, it successfully provides a highly functional platform for learning and experimenting with SCADA and DCS technologies.

B. Future Work

There are several avenues in which this project can be improved and expanded upon in the future, besides software bug fixing.

1) *Ethernet Connected DCS*: A way that the communication protocol of the code-base could be expanded would be to add the capability for Ethernet-driven DCS communication as opposed to USB serial. This would allow the Arduino Ethernet Shield to be utilized.

2) *Bluetooth Connected DCS*: Another way that the communication protocol of the code-base could be expanded would be to add the capability for Bluetooth-driven DCS communication. This would similarly allow the Arduino Bluetooth module to be utilized.

3) *Wifi Connected DCS*: Another similar way that the communication protocol of the code-base could be expanded would be to add the capability for Wifi-driven DCS communication. This would represent all three layers of control being separated on a local network, and would require significant code refactoring and hardware addition.

4) *Raspberry Pie DCS*: The Raspberry Pi computers are much more powerful and much faster than the Arduino AVR boards, these could be used as DCS controllers instead of SCADA controllers in systems that need faster response or contain more information (more complex equipment).

5) *Addition of more Code Blocks*: The selection of code blocks is limited due to this software being experimental, expanding the default library of code blocks would be useful.

6) *Code Block Editor*: An editor in the CLI and GUI that would allow a user to make or edit code-blocks would also alleviate the issues of code-block sparsity in a more custom way.